

Sistema de Transferencias entre Tiendas - Modelo Prescriptivo

Objetivo

Desarrollar un modelo prescriptivo que optimice las transferencias de stock entre tiendas para minimizar costos logísticos (transferencia, rotura de stock, obsolescencia), utilizando programación lineal y forecasting de demanda.

Enfoque

- 1. **Forecasting de demanda** usando series temporales
- 2. **Identificación de estados** (tiendas con déficit/exceso proyectado)
- 3. **Optimización con PuLP** para determinar transferencias óptimas
- 4. **Evaluación en 5 escenarios** de costos logísticos diferentes

```
In [1]: import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

# Para optimización
from pulp import *

# Para forecasting
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Para visualización
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Configuración de visualización
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)

print("Bibliotecas importadas correctamente")
```

Bibliotecas importadas correctamente

1. Datos Maestros de Productos

```
In [2]: productos = pd.read_csv("productos.csv")
```

```
In [3]: productos.sample(5)
```

Out[3]:

	sku	categoria	subcategoria	talla	color	costo_unitario	precio_venta	proveedor_id	lead_time_dias	cantidad_minima_pedido
77	HM000078	Pantalones	Temporada	M	Blanco	16.23	60.27	PROV078	17	100
2	HM000003	Abrigos	Basico	L	Blanco	40.50	68.38	PROV003	12	100
5	HM000006	Camisetas	Temporada	XS	Negro	26.72	77.14	PROV006	20	500
14	HM000015	Pantalones	Basico	S	Negro	32.71	87.54	PROV015	15	200
23	HM000024	Camisetas	Basico	S	Rojo	20.21	80.55	PROV024	9	50

2. Datos Históricos de Ventas (365 días)

```
In [4]: ventas = pd.read_csv('ventas.csv')
```

```
In [5]: ventas.sample(5)
```

Out[5]:

	fecha	tienda	sku	cantidad_vendida	precio_venta_unitario	descuento_aplicado	venta_neta
416488	2023-06-11	TIENDA007	HM000042	1	66.68	0.0	66.68
281904	2023-04-20	TIENDA009	HM000024	3	98.10	0.1	264.87
915999	2023-12-23	TIENDA004	HM000098	1	55.90	0.0	55.90
514898	2023-07-20	TIENDA006	HM000070	2	87.50	0.0	175.00
445461	2023-06-23	TIENDA010	HM000004	1	98.92	0.0	98.92

3. Datos de Inventario Actual

In [6]:

```
inventario = pd.read_csv('inventario.csv')
```

In [7]:

```
inventario.sample(5)
```

Out[7]:

	tienda	sku	stock_actual	stock_seguridad_actual	stock_en_transito	fecha_ultima_reposicion
252	TIENDA003	HM000018	87	19	6	2024-01-09
423	TIENDA005	HM000043	63	24	18	2024-01-13
330	TIENDA004	HM000012	81	21	47	2024-01-08
20	TIENDA001	HM000056	46	17	44	2023-12-23
175	TIENDA002	HM000058	62	20	30	2024-01-01

4. Datos de Escenarios con Costos Logísticos

In [8]:

```
Costos_Logisticos = pd.read_csv('Costos_Logisticos.csv')
```

In [9]:

```
Costos_Logisticos
```

Out[9]:

	Escenario	costo_pedido	costo_mantenimiento_anual	costo_almacenamiento_m2	costo_transferencia_tienda	costo_rotura_stock	costo_otro
0	Escenario 1	10.0	0.15	80.0	3.0	0.20	
1	Escenario 2	40.0	0.20	100.0	4.0	0.25	
2	Escenario 3	50.0	0.25	120.0	5.0	0.30	
3	Escenario 4	55.0	0.30	130.0	6.0	0.35	
4	Escenario 5	60.0	0.35	140.0	7.0	0.38	

In [10]:

```
# Instalación de bibliotecas necesarias
# !pip install pulp statsmodels prophet plotly streamlit scikit-learn
```

Paso 3: Resumen de Datos

In [11]:

```
# Convertir fecha a datetime
ventas['fecha'] = pd.to_datetime(ventas['fecha'])

# Información general
print("="*80)
print("RESUMEN DE DATOS")
print("="*80)
print(f"\nProductos: {len(productos)} SKUs")
print(f"Tiendas: {inventario['tienda'].nunique()} tiendas")
print(f"Ventas: {len(ventas):,} registros")
print(f"Período ventas: {ventas['fecha'].min()} a {ventas['fecha'].max()}")
print(f"Escenarios de costos: {len(Costos_Logisticos)}")

print("\n" + "="*80)
print("TOP 10 PRODUCTOS MÁS VENDIDOS (por cantidad)")
print("="*80)
top_products = ventas.groupby('sku')['cantidad_vendida'].sum().sort_values(ascending=False).head(10)
top_products_info = productos[productos['sku'].isin(top_products.index)][['sku', 'categoria', 'subcategoria', 'precio_venta']]
top_products_df = pd.DataFrame({
    'SKU': top_products.index,
    'Cantidad_Vendida': top_products.values
}).merge(top_products_info, left_on='SKU', right_on='sku').drop('sku', axis=1)
print(top_products_df.to_string(index=False))
```

```
print("\n" + "="*80)
print("DISTRIBUCIÓN DE VENTAS POR TIENDA")
print("="*80)
ventas_tienda = ventas.groupby('tienda')['cantidad_vendida'].sum().sort_values(ascending=False)
print(ventas_tienda.to_string())
```

RESUMEN DE DATOS

Productos: 100 SKUs
Tiendas: 6 tiendas
Ventas: 940,550 registros
Período ventas: 2023-01-01 00:00:00 a 2023-12-31 00:00:00
Escenarios de costos: 5

TOP 10 PRODUCTOS MÁS VENDIDOS (por cantidad)

SKU	Cantidad_Vendida	categoria	subcategoria	precio_venta
HM000035	16726	Camisetas	Temporada	98.11
HM000003	16529	Abrigos	Basico	68.38
HM000057	16501	Accesorios	Premium	18.99
HM000095	16498	Vestidos	Temporada	76.75
HM000041	16418	Accesorios	Premium	71.15
HM000033	16416	Camisetas	Premium	82.05
HM000014	16412	Accesorios	Basico	46.53
HM000039	16386	Pantalones	Premium	69.16
HM000062	16385	Pantalones	Basico	57.72
HM000005	16324	Accesorios	Temporada	56.41

DISTRIBUCIÓN DE VENTAS POR TIENDA

tienda	
TIENDA007	152418
TIENDA005	151610
TIENDA001	151185
TIENDA008	151026
TIENDA003	150959
TIENDA010	150940
TIENDA006	150765
TIENDA002	150570
TIENDA004	150187
TIENDA009	149432

Selección de Subset para Modelo (Top SKUs y Tiendas Principales)

Para manejar la complejidad computacional, trabajaremos con:

- **Top 10 SKUs** más vendidos
- **Top 5 tiendas** con mayor volumen de ventas

```
In [12]: # Seleccionar Top 10 SKUs y Top 5 Tiendas
TOP_N_SKUS = 10
TOP_N_TIENDAS = 5

# Top SKUs
top_skus = ventas.groupby('sku')['cantidad_vendida'].sum().nlargest(TOP_N_SKUS).index.tolist()

# Top Tiendas
top_tiendas = ventas.groupby('tienda')['cantidad_vendida'].sum().nlargest(TOP_N_TIENDAS).index.tolist()

# Filtrar datos
ventas_filtered = ventas[ventas['sku'].isin(top_skus) & ventas['tienda'].isin(top_tiendas)].copy()
inventario_filtered = inventario[inventario['sku'].isin(top_skus) & inventario['tienda'].isin(top_tiendas)].copy()
productos_filtered = productos[productos['sku'].isin(top_skus)].copy()

print(f"Dataset reducido:")
print(f" - SKUs seleccionados: {len(top_skus)}")
print(f" - Tiendas seleccionadas: {len(top_tiendas)}")
print(f" - Registros de ventas: {len(ventas_filtered):,}")
print(f"\nSKUs: {top_skus}")
print(f"Tiendas: {top_tiendas}")
```

Dataset reducido:

- SKUs seleccionados: 10
- Tiendas seleccionadas: 5
- Registros de ventas: 47,654

SKUs: ['HM000035', 'HM000003', 'HM000057', 'HM000095', 'HM000041', 'HM000033', 'HM000014', 'HM000039', 'HM000062', 'HM000005']
Tiendas: ['TIENDA007', 'TIENDA005', 'TIENDA001', 'TIENDA008', 'TIENDA003']

Paso 4: Forecasting de Demanda

Utilizaremos modelos de series temporales para proyectar la demanda de los próximos 30 días para cada combinación de SKU-Tienda.

```
In [13]: # Preparar series temporales - agregación diaria
ventas_diarias = ventas_filtered.groupby(['fecha', 'tienda', 'sku'])['cantidad_vendida'].sum().reset_index()

# Crear un índice completo de fechas para todas las combinaciones
fecha_min = ventas_diarias['fecha'].min()
fecha_max = ventas_diarias['fecha'].max()
fechas_completas = pd.date_range(start=fecha_min, end=fecha_max, freq='D')

print(f"Rango de fechas: {fecha_min} a {fecha_max}")
print(f"Total de días: {len(fechas_completas)}")
print(f"Combinaciones SKU-Tienda: {len(top_skus) * len(top_tiendas)}")
```

Rango de fechas: 2023-01-01 00:00:00 a 2023-12-31 00:00:00

Total de días: 365

Combinaciones SKU-Tienda: 50

```
In [14]: def forecast_demand_simple(serie_temporal, periods=30):
    """
    Realiza forecasting simple usando media móvil y tendencia.

    Args:
        serie_temporal: Series temporal de demanda histórica
        periods: Número de períodos a proyectar

    Returns:
        Demanda proyectada total para el período
    """
    if len(serie_temporal) < 7:
        # Si hay pocos datos, usar promedio
        return serie_temporal.mean() * periods if len(serie_temporal) > 0 else 0

    # Calcular media móvil de últimos 30 días
    ventana = min(30, len(serie_temporal))
    media_movil = serie_temporal.tail(ventana).mean()

    # Calcular tendencia
    if len(serie_temporal) >= 14:
        primera_mitad = serie_temporal[:len(serie_temporal)//2].mean()
        segunda_mitad = serie_temporal[len(serie_temporal)//2:].mean()
        tendencia = (segunda_mitad - primera_mitad) / (len(serie_temporal)//2)
    else:
        tendencia = 0

    # Proyección ajustada por tendencia
    demanda_proyectada = media_movil * periods + (tendencia * periods * (periods + 1) / 2)

    return max(0, demanda_proyectada) # No permitir demanda negativa

# Calcular demanda proyectada para cada combinación SKU-Tienda
demanda_proyectada_dict = {}
FORECAST_DAYS = 30

print("Calculando forecasting de demanda...")
print("="*80)

for tienda in top_tiendas:
    for sku in top_skus:
        # Filtrar ventas para esta combinación
        ventas_ts = ventas_diarias[
            (ventas_diarias['tienda'] == tienda) &
            (ventas_diarias['sku'] == sku)
        ].set_index('fecha')['cantidad_vendida'].sort_index()

        # Rellenar días sin ventas con 0
        ventas_ts = ventas_ts.reindex(fechas_completas, fill_value=0)

        # Calcular forecast
        demanda_proj = forecast_demand_simple(ventas_ts, FORECAST_DAYS)
        demanda_proyectada_dict[(tienda, sku)] = demanda_proj

    print(f"{tienda} - {sku}: Demanda histórica promedio = {ventas_ts.mean():.2f}, "
          f"Demanda proyectada (30d) = {demanda_proj:.2f}")

print(f"\nForecasting completado para {len(demanda_proyectada_dict)} combinaciones")
```

Calculando forecasting de demanda...

=====

TIENDA007	-	HM000035:	Demanda histórica promedio = 4.41,	Demanda proyectada (30d) = 125.79
TIENDA007	-	HM000003:	Demanda histórica promedio = 4.64,	Demanda proyectada (30d) = 114.37
TIENDA007	-	HM000057:	Demanda histórica promedio = 4.44,	Demanda proyectada (30d) = 131.03
TIENDA007	-	HM000095:	Demanda histórica promedio = 4.81,	Demanda proyectada (30d) = 123.65
TIENDA007	-	HM000041:	Demanda histórica promedio = 4.64,	Demanda proyectada (30d) = 145.50
TIENDA007	-	HM000033:	Demanda histórica promedio = 4.68,	Demanda proyectada (30d) = 214.48
TIENDA007	-	HM000014:	Demanda histórica promedio = 4.43,	Demanda proyectada (30d) = 173.03
TIENDA007	-	HM000039:	Demanda histórica promedio = 4.44,	Demanda proyectada (30d) = 145.57
TIENDA007	-	HM000062:	Demanda histórica promedio = 4.32,	Demanda proyectada (30d) = 197.09
TIENDA007	-	HM000005:	Demanda histórica promedio = 4.39,	Demanda proyectada (30d) = 144.33
TIENDA005	-	HM000035:	Demanda histórica promedio = 4.62,	Demanda proyectada (30d) = 143.05
TIENDA005	-	HM000003:	Demanda histórica promedio = 4.77,	Demanda proyectada (30d) = 134.77
TIENDA005	-	HM000057:	Demanda histórica promedio = 4.31,	Demanda proyectada (30d) = 103.05
TIENDA005	-	HM000095:	Demanda histórica promedio = 4.16,	Demanda proyectada (30d) = 98.59
TIENDA005	-	HM000041:	Demanda histórica promedio = 4.80,	Demanda proyectada (30d) = 106.74
TIENDA005	-	HM000033:	Demanda histórica promedio = 4.65,	Demanda proyectada (30d) = 206.92
TIENDA005	-	HM000014:	Demanda histórica promedio = 4.42,	Demanda proyectada (30d) = 211.88
TIENDA005	-	HM000039:	Demanda histórica promedio = 4.34,	Demanda proyectada (30d) = 118.93
TIENDA005	-	HM000062:	Demanda histórica promedio = 4.36,	Demanda proyectada (30d) = 167.33
TIENDA005	-	HM000005:	Demanda histórica promedio = 4.65,	Demanda proyectada (30d) = 126.29
TIENDA001	-	HM000035:	Demanda histórica promedio = 4.76,	Demanda proyectada (30d) = 127.74
TIENDA001	-	HM000003:	Demanda histórica promedio = 4.64,	Demanda proyectada (30d) = 106.17
TIENDA001	-	HM000057:	Demanda histórica promedio = 4.70,	Demanda proyectada (30d) = 122.62
TIENDA001	-	HM000095:	Demanda histórica promedio = 4.26,	Demanda proyectada (30d) = 90.11
TIENDA001	-	HM000041:	Demanda histórica promedio = 4.21,	Demanda proyectada (30d) = 105.14
TIENDA001	-	HM000033:	Demanda histórica promedio = 4.41,	Demanda proyectada (30d) = 200.27
TIENDA001	-	HM000014:	Demanda histórica promedio = 4.35,	Demanda proyectada (30d) = 165.82
TIENDA001	-	HM000039:	Demanda histórica promedio = 4.62,	Demanda proyectada (30d) = 125.90
TIENDA001	-	HM000062:	Demanda histórica promedio = 4.49,	Demanda proyectada (30d) = 191.41
TIENDA001	-	HM000005:	Demanda histórica promedio = 4.58,	Demanda proyectada (30d) = 156.53
TIENDA008	-	HM000035:	Demanda histórica promedio = 4.87,	Demanda proyectada (30d) = 128.79
TIENDA008	-	HM000003:	Demanda histórica promedio = 4.50,	Demanda proyectada (30d) = 123.76
TIENDA008	-	HM000057:	Demanda histórica promedio = 4.53,	Demanda proyectada (30d) = 143.88
TIENDA008	-	HM000095:	Demanda histórica promedio = 4.73,	Demanda proyectada (30d) = 105.89
TIENDA008	-	HM000041:	Demanda histórica promedio = 4.26,	Demanda proyectada (30d) = 99.72
TIENDA008	-	HM000033:	Demanda histórica promedio = 4.38,	Demanda proyectada (30d) = 148.87
TIENDA008	-	HM000014:	Demanda histórica promedio = 4.45,	Demanda proyectada (30d) = 177.04
TIENDA008	-	HM000039:	Demanda histórica promedio = 4.44,	Demanda proyectada (30d) = 126.28
TIENDA008	-	HM000062:	Demanda histórica promedio = 4.50,	Demanda proyectada (30d) = 215.52
TIENDA008	-	HM000005:	Demanda histórica promedio = 4.41,	Demanda proyectada (30d) = 118.17
TIENDA003	-	HM000035:	Demanda histórica promedio = 4.69,	Demanda proyectada (30d) = 119.84
TIENDA003	-	HM000003:	Demanda histórica promedio = 4.38,	Demanda proyectada (30d) = 110.20
TIENDA003	-	HM000057:	Demanda histórica promedio = 4.29,	Demanda proyectada (30d) = 88.06
TIENDA003	-	HM000095:	Demanda histórica promedio = 4.75,	Demanda proyectada (30d) = 89.56
TIENDA003	-	HM000041:	Demanda histórica promedio = 4.38,	Demanda proyectada (30d) = 126.30
TIENDA003	-	HM000033:	Demanda histórica promedio = 4.64,	Demanda proyectada (30d) = 190.18
TIENDA003	-	HM000014:	Demanda histórica promedio = 4.64,	Demanda proyectada (30d) = 201.71
TIENDA003	-	HM000039:	Demanda histórica promedio = 4.84,	Demanda proyectada (30d) = 116.65
TIENDA003	-	HM000062:	Demanda histórica promedio = 4.81,	Demanda proyectada (30d) = 181.15
TIENDA003	-	HM000005:	Demanda histórica promedio = 4.64,	Demanda proyectada (30d) = 118.20

Forecasting completado para 50 combinaciones

Paso 5: Identificación de Estados de Tiendas (Déficit/Exceso) - Análisis de Estado de Inventario

Clasificaremos cada tienda según su estado proyectado de inventario.

```
In [15]: # Crear diccionario de estados de inventario
estados_inventario = {}

print("Análisis de Estados de Inventario")
print("="*80)
print(f"{'Tienda':<12} {'SKU':<12} {'Stock':<8} {'SS':<8} {'Demanda':<10} {'Estado':<15}")
print("="*80)

tiendas_deficit = [] # Lista de (tienda, sku) con déficit
tiendas_exceso = [] # Lista de (tienda, sku) con exceso

for tienda in top_tiendas:
    for sku in top_skus:
        # Obtener datos de inventario
        inv_row = inventario_filtered[
            (inventario_filtered['tienda'] == tienda) &
            (inventario_filtered['sku'] == sku)
        ]

        if len(inv_row) == 0:
            # No hay inventario registrado, asumir stock 0
            stock_actual = 0
            stock_seguridad = 0
        else:
            stock_actual = inv_row['stock_actual'].values[0]
```

```

        stock_seguridad = inv_row['stock_seguridad_actual'].values[0]

# Obtener demanda proyectada
        demanda_proj = demanda_proyectada_dict.get((tienda, sku), 0)

# Calcular necesidad total
        necesidad_total = stock_seguridad + demanda_proj

# Determinar estado
        diferencia = stock_actual - necesidad_total

        if diferencia < 0:
            estado = "DÉFICIT"
            tiendas_deficit.append((tienda, sku, abs(diferencia)))
        elif diferencia > demanda_proj * 0.5: # Exceso si tiene más del 50% de demanda proyectada de sobra
            estado = "EXCESO"
            tiendas_exceso.append((tienda, sku, diferencia))
        else:
            estado = "EQUILIBRADO"

        estados_inventario[(tienda, sku)] = {
            'stock_actual': stock_actual,
            'stock_seguridad': stock_seguridad,
            'demanda_proyectada': demanda_proj,
            'necesidad_total': necesidad_total,
            'diferencia': diferencia,
            'estado': estado
        }

    print(f"{tienda:<12} {sku:<12} {stock_actual:<8.0f} {stock_seguridad:<8.0f} "
          f"{demanda_proj:<10.1f} {estado:<15}")

print("="*80)
print(f"\nTiendas con DÉFICIT proyectado: {len(tiendas_deficit)}")
print(f"Tiendas con EXCESO proyectado: {len(tiendas_exceso)}")
print(f"Tiendas en EQUILIBRIO: {len(estados_inventario) - len(tiendas_deficit) - len(tiendas_exceso)}")

```


Análisis de Estados de Inventario					
Tienda	SKU	Stock	SS	Demanda	Estado
TIENDA007	HM000035	0	0	125.8	DÉFICIT
TIENDA007	HM000003	0	0	114.4	DÉFICIT
TIENDA007	HM000057	0	0	131.0	DÉFICIT
TIENDA007	HM000095	0	0	123.6	DÉFICIT
TIENDA007	HM000041	0	0	145.5	DÉFICIT
TIENDA007	HM000033	0	0	214.5	DÉFICIT
TIENDA007	HM000014	0	0	173.0	DÉFICIT
TIENDA007	HM000039	0	0	145.6	DÉFICIT
TIENDA007	HM000062	0	0	197.1	DÉFICIT
TIENDA007	HM000005	0	0	144.3	DÉFICIT
TIENDA005	HM000035	21	28	143.1	DÉFICIT
TIENDA005	HM000003	52	28	134.8	DÉFICIT
TIENDA005	HM000057	7	21	103.1	DÉFICIT
TIENDA005	HM000095	39	32	98.6	DÉFICIT
TIENDA005	HM000041	27	14	106.7	DÉFICIT
TIENDA005	HM000033	37	17	206.9	DÉFICIT
TIENDA005	HM000014	58	12	211.9	DÉFICIT
TIENDA005	HM000039	52	23	118.9	DÉFICIT
TIENDA005	HM000062	16	19	167.3	DÉFICIT
TIENDA005	HM000005	19	21	126.3	DÉFICIT
TIENDA001	HM000035	14	24	127.7	DÉFICIT
TIENDA001	HM000003	87	18	106.2	DÉFICIT
TIENDA001	HM000057	15	25	122.6	DÉFICIT
TIENDA001	HM000095	69	27	90.1	DÉFICIT
TIENDA001	HM000041	2	16	105.1	DÉFICIT
TIENDA001	HM000033	73	12	200.3	DÉFICIT
TIENDA001	HM000014	72	19	165.8	DÉFICIT
TIENDA001	HM000039	50	20	125.9	DÉFICIT
TIENDA001	HM000062	91	21	191.4	DÉFICIT
TIENDA001	HM000005	57	32	156.5	DÉFICIT
TIENDA008	HM000035	0	0	128.8	DÉFICIT
TIENDA008	HM000003	0	0	123.8	DÉFICIT
TIENDA008	HM000057	0	0	143.9	DÉFICIT
TIENDA008	HM000095	0	0	105.9	DÉFICIT
TIENDA008	HM000041	0	0	99.7	DÉFICIT
TIENDA008	HM000033	0	0	148.9	DÉFICIT
TIENDA008	HM000014	0	0	177.0	DÉFICIT
TIENDA008	HM000039	0	0	126.3	DÉFICIT
TIENDA008	HM000062	0	0	215.5	DÉFICIT
TIENDA008	HM000005	0	0	118.2	DÉFICIT
TIENDA003	HM000035	53	18	119.8	DÉFICIT
TIENDA003	HM000003	12	22	110.2	DÉFICIT
TIENDA003	HM000057	90	20	88.1	DÉFICIT
TIENDA003	HM000095	83	21	89.6	DÉFICIT
TIENDA003	HM000041	94	30	126.3	DÉFICIT
TIENDA003	HM000033	61	23	190.2	DÉFICIT
TIENDA003	HM000014	49	14	201.7	DÉFICIT
TIENDA003	HM000039	93	15	116.7	DÉFICIT
TIENDA003	HM000062	38	16	181.1	DÉFICIT
TIENDA003	HM000005	5	24	118.2	DÉFICIT

Tiendas con DÉFICIT proyectado: 50
Tiendas con EXCESO proyectado: 0
Tiendas en EQUILIBRIO: 0

Paso 6: Modelo de Optimización - Sistema de Transferencias

Formulación Matemática:

Variables de Decisión:

- x_{ijk} : Cantidad del SKU k a transferir de tienda i a tienda j (entera, no negativa)

Función Objetivo: Minimizar:

$$Z = \sum_{i,j,k} (C_{trans} \cdot x_{ijk}) + \sum_j (C_{rotura} \cdot Rotura_j) + \sum_i (C_{obs} \cdot Obsolescencia_i)$$

Restricciones:

- Stock post-transferencia en origen no debe caer bajo stock de seguridad
- Transferencias no pueden exceder stock disponible en origen
- Satisfacer demanda proyectada en destino
- Variables no negativas y enteras

```
In [16]: def optimizar_transferencias(escenario_costos, nombre_escenario="Escenario"):  
        """  
        Optimiza las transferencias entre tiendas para un escenario de costos dado.
```

```

Args:
    escenario_costos: Dict con costos del escenario
    nombre_escenario: Nombre del escenario para identificación

Returns:
    Tuple: (modelo, resultados_df, metricas_dict)
    """

# Extraer costos del escenario
c_transferencia = escenario_costos['costo_transferencia_tienda']
c_rotura = escenario_costos['costo_rotura_stock']
c_obsolescencia = escenario_costos['costo_obsolescencia']

print(f"\n{'='*80}")
print(f"OPTIMIZANDO: {nombre_escenario}")
print(f"{'='*80}")
print(f"Costo transferencia: ${c_transferencia:.2f} por unidad")
print(f"Costo rotura stock: ${c_rotura:.2f} por unidad faltante")
print(f"Costo obsolescencia: ${c_obsolescencia:.2f} por unidad en exceso")

# Crear modelo
modelo = LpProblem(f"Optimizacion_Transferencias_{nombre_escenario}", LpMinimize)

# Variables de decisión: transferencias
transferencias = {}
for i in top_tiendas:
    for j in top_tiendas:
        if i != j: # No transferir a la misma tienda
            for k in top_skus:
                var_name = f"Trans_{i}_{j}_{k}"
                transferencias[(i, j, k)] = LpVariable(var_name, lowBound=0, cat='Integer')

# Variables auxiliares para penalidades
rotura_vars = {} # Déficit no cubierto
exceso_vars = {} # Exceso que queda

for tienda in top_tiendas:
    for sku in top_skus:
        rotura_vars[(tienda, sku)] = LpVariable(f"Rotura_{tienda}_{sku}", lowBound=0, cat='Continuous')
        exceso_vars[(tienda, sku)] = LpVariable(f"Exceso_{tienda}_{sku}", lowBound=0, cat='Continuous')

# FUNCIÓN OBJETIVO
# Minimizar: Costos de transferencia + Penalidades por rotura + Penalidades por obsolescencia

costo_transferencia = lpSum([
    c_transferencia * transferencias[(i, j, k)]
    for i in top_tiendas for j in top_tiendas if i != j
    for k in top_skus
])

penalidad_rotura = lpSum([
    c_rotura * rotura_vars[(tienda, sku)]
    for tienda in top_tiendas for sku in top_skus
])

penalidad_exceso = lpSum([
    c_obsolescencia * exceso_vars[(tienda, sku)]
    for tienda in top_tiendas for sku in top_skus
])

modelo += costo_transferencia + penalidad_rotura + penalidad_exceso, "Costo_Total"

# RESTRICCIONES

# Para cada tienda y SKU
for tienda in top_tiendas:
    for sku in top_skus:
        estado = estados_inventario[(tienda, sku)]
        stock_actual = estado['stock_actual']
        stock_seguridad = estado['stock_seguridad']
        demanda_proj = estado['demanda_proyectada']

        # Total salidas de esta tienda
        salidas = lpSum([transferencias[(tienda, j, sku)]
                        for j in top_tiendas if j != tienda])

        # Total entradas a esta tienda
        entradas = lpSum([transferencias[(i, tienda, sku)]
                        for i in top_tiendas if i != tienda])

        # Stock final = stock inicial - salidas + entradas
        stock_final = stock_actual - salidas + entradas

        # 1. Stock origen no debe caer bajo stock de seguridad después de transferencias
        modelo += stock_final >= stock_seguridad, f"StockSeguridad_{tienda}_{sku}"

        # 2. Calcular rotura (déficit no cubierto)

```



```

necesidad = stock_seguridad + demanda_proj
modelo += rotura_vars[(tienda, sku)] >= necesidad - stock_final, f"Rotura_{tienda}_{sku}"

# 3. Calcular exceso
modelo += exceso_vars[(tienda, sku)] >= stock_final - necesidad, f"Exceso_{tienda}_{sku}"

# Resolver modelo
print(f"\nResolviendo modelo de optimización...")
modelo.solve(PULP_CBC_CMD(msg=0))

# Verificar estado de la solución
estado_solucion = LpStatus[modelo.status]
print(f"Estado de la solución: {estado_solucion}")

if modelo.status != 1: # 1 = Optimal
    print(f"Advertencia: La solución no es óptima")
    return modelo, pd.DataFrame(), {}

# Extraer resultados
resultados = []
costo_trans_total = 0
costo_rotura_total = 0
costo_exceso_total = 0

for (i, j, k), var in transferencias.items():
    cantidad = var.varValue
    if cantidad is not None and cantidad > 0.5: # Umbral para considerar transferencia
        costo_trans = cantidad * c_transferencia
        costo_trans_total += costo_trans

    resultados.append({
        'Tienda_Origen': i,
        'Tienda_Destino': j,
        'SKU': k,
        'Cantidad': int(cantidad),
        'Costo_Transferencia': costo_trans
    })

# Calcular costos de rotura y exceso
for tienda in top_tiendas:
    for sku in top_skus:
        rotura = rotura_vars[(tienda, sku)].varValue
        exceso = exceso_vars[(tienda, sku)].varValue

        if rotura is not None and rotura > 0:
            costo_rotura_total += rotura * c_rotura

        if exceso is not None and exceso > 0:
            costo_exceso_total += exceso * c_obsolescencia

resultados_df = pd.DataFrame(resultados)

metricas = {
    'Costo_Transferencias': costo_trans_total,
    'Costo_Rotura': costo_rotura_total,
    'Costo_Exceso': costo_exceso_total,
    'Costo_Total': value(modelo.objective),
    'Num_Transferencias': len(resultados_df),
    'Unidades_Transferidas': resultados_df['Cantidad'].sum() if len(resultados_df) > 0 else 0
}

print(f"\nRESULTADOS FINANCIEROS:")
print(f" - Costo de transferencias: ${metricas['Costo_Transferencias']:.2f}")
print(f" - Costo de rotura de stock: ${metricas['Costo_Rotura']:.2f}")
print(f" - Costo de exceso/obsolescencia: ${metricas['Costo_Exceso']:.2f}")
print(f" - COSTO TOTAL: ${metricas['Costo_Total']:.2f}")
print(f"\nOPERACIONES:")
print(f" - Número de transferencias: {metricas['Num_Transferencias']}")
print(f" - Unidades totales transferidas: {metricas['Unidades_Transferidas']:.0f}")

return modelo, resultados_df, metricas

print("Función de optimización definida")

```

Función de optimización definida

Paso 7: Ejecución del Modelo para los 5 Escenarios

Ejecutaremos el modelo de optimización para cada uno de los 5 escenarios de costos logísticos.

```

In [17]: # Ejecutar optimización para cada escenario
resultados_por_escenario = {}
metricas_por_escenario = {}
modelos_por_escenario = {}

for idx, row in Costos_Logisticos.iterrows():

```

```
escenario_nombre = row['Escenario']

escenario_costos = {
    'costo_transferencia_tienda': row['costo_transferencia_tienda'],
    'costo_rotura_stock': row['costo_rotura_stock'],
    'costo_obsolescencia': row['costo_obsolescencia']
}

modelo, resultados_df, metricas = optimizar_transferencias(escenario_costos, escenario_nombre)

resultados_por_escenario[escenario_nombre] = resultados_df
metricas_por_escenario[escenario_nombre] = metricas
modelos_por_escenario[escenario_nombre] = modelo

print("\n" + "="*80)
print("OPTIMIZACIÓN COMPLETADA PARA TODOS LOS ESCENARIOS")
print("="*80)
```

```
=====
OPTIMIZANDO: Escenario 1
=====
Costo transferencia: $3.00 por unidad
Costo rotura stock: $0.20 por unidad faltante
Costo obsolescencia: $0.09 por unidad en exceso

Resolviendo modelo de optimización...
Estado de la solución: Optimal

RESULTADOS FINANCIEROS:
- Costo de transferencias: $267.00
- Costo de rotura de stock: $1,249.97
- Costo de exceso/obsolescencia: $0.00
- COSTO TOTAL: $1,516.97

OPERACIONES:
- Número de transferencias: 9
- Unidades totales transferidas: 89

=====
OPTIMIZANDO: Escenario 2
=====
Costo transferencia: $4.00 por unidad
Costo rotura stock: $0.25 por unidad faltante
Costo obsolescencia: $0.12 por unidad en exceso

Resolviendo modelo de optimización...
Estado de la solución: Optimal

RESULTADOS FINANCIEROS:
- Costo de transferencias: $356.00
- Costo de rotura de stock: $1,562.47
- Costo de exceso/obsolescencia: $0.00
- COSTO TOTAL: $1,918.47

OPERACIONES:
- Número de transferencias: 9
- Unidades totales transferidas: 89

=====
OPTIMIZANDO: Escenario 3
=====
Costo transferencia: $5.00 por unidad
Costo rotura stock: $0.30 por unidad faltante
Costo obsolescencia: $0.15 por unidad en exceso

Resolviendo modelo de optimización...
Estado de la solución: Optimal

RESULTADOS FINANCIEROS:
- Costo de transferencias: $445.00
- Costo de rotura de stock: $1,874.96
- Costo de exceso/obsolescencia: $0.00
- COSTO TOTAL: $2,319.96

OPERACIONES:
- Número de transferencias: 9
- Unidades totales transferidas: 89

=====
OPTIMIZANDO: Escenario 4
=====
Costo transferencia: $6.00 por unidad
Costo rotura stock: $0.35 por unidad faltante
Costo obsolescencia: $0.18 por unidad en exceso

Resolviendo modelo de optimización...
Estado de la solución: Optimal

RESULTADOS FINANCIEROS:
- Costo de transferencias: $534.00
- Costo de rotura de stock: $2,187.45
- Costo de exceso/obsolescencia: $0.00
- COSTO TOTAL: $2,721.45

OPERACIONES:
- Número de transferencias: 9
- Unidades totales transferidas: 89

=====
OPTIMIZANDO: Escenario 5
=====
Costo transferencia: $7.00 por unidad
Costo rotura stock: $0.38 por unidad faltante
Costo obsolescencia: $0.20 por unidad en exceso
```

Resolviendo modelo de optimización...
Estado de la solución: Optimal

- RESULTADOS FINANCIEROS:
- Costo de transferencias: \$623.00
 - Costo de rotura de stock: \$2,374.95
 - Costo de exceso/obsolescencia: \$0.00
 - COSTO TOTAL: \$2,997.95

- OPERACIONES:
- Número de transferencias: 9
 - Unidades totales transferidas: 89

=====

OPTIMIZACIÓN COMPLETADA PARA TODOS LOS ESCENARIOS

=====

Paso 8: Análisis Comparativo de Escenarios

```
In [18]: # Crear tabla comparativa de métricas
comparacion_df = pd.DataFrame(metricas_por_escenario).T
comparacion_df.index.name = 'Escenario'
comparacion_df = comparacion_df.reset_index()

# Mostrar tabla
print("\n" + "="*80)
print("COMPARACIÓN DE ESCENARIOS")
print("="*80)
print(comparacion_df.to_string(index=False))

# Identificar mejor escenario
mejor_escenario = comparacion_df.loc[comparacion_df['Costo_Total'].idxmin(), 'Escenario']
print(f"\nMEJOR ESCENARIO: {mejor_escenario} (Menor costo total)")

# Guardar resultados
comparacion_df.to_csv('comparacion_escenarios.csv', index=False)
print("\nComparación guardada en 'comparacion_escenarios.csv'")
```

=====

COMPARACIÓN DE ESCENARIOS

=====

Escenario	Costo_Transferencias	Costo_Rotura	Costo_Exceso	Costo_Total	Num_Transferencias	Unidades_Transferidas
Escenario 1	267.0	1249.972716	0.0	1516.972716	9.0	89.0
Escenario 2	356.0	1562.465894	0.0	1918.465895	9.0	89.0
Escenario 3	445.0	1874.959073	0.0	2319.959073	9.0	89.0
Escenario 4	534.0	2187.452252	0.0	2721.452252	9.0	89.0
Escenario 5	623.0	2374.948160	0.0	2997.948160	9.0	89.0

MEJOR ESCENARIO: Escenario 1 (Menor costo total)

Comparación guardada en 'comparacion_escenarios.csv'

```
In [19]: # Visualización comparativa
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=('Costo Total por Escenario',
                    'Desglose de Costos',
                    'Número de Transferencias',
                    'Unidades Transferidas'),
    specs=[[{'type': 'bar'}, {'type': 'bar'}],
           [{'type': 'bar'}, {'type': 'bar'}]]
)

# Gráfico 1: Costo Total
fig.add_trace(
    go.Bar(x=comparacion_df['Escenario'],
            y=comparacion_df['Costo_Total'],
            name='Costo Total',
            marker_color='indianred'),
    row=1, col=1
)

# Gráfico 2: Desglose de costos
escenarios = comparacion_df['Escenario'].tolist()
fig.add_trace(
    go.Bar(x=escenarios, y=comparacion_df['Costo_Transferencias'],
            name='Transferencias', marker_color='lightblue'),
    row=1, col=2
)
fig.add_trace(
    go.Bar(x=escenarios, y=comparacion_df['Costo_Rotura'],
            name='Rotura', marker_color='orange'),
    row=1, col=2
)
fig.add_trace(
    go.Bar(x=escenarios, y=comparacion_df['Costo_Exceso'],
```

```

        name='Exceso', marker_color='green'),
        row=1, col=2
    )

# Gráfico 3: Número de transferencias
fig.add_trace(
    go.Bar(x=comparacion_df['Escenario'],
           y=comparacion_df['Num_Transferencias'],
           name='Num. Transferencias',
           marker_color='purple'),
        row=2, col=1
    )

# Gráfico 4: Unidades transferidas
fig.add_trace(
    go.Bar(x=comparacion_df['Escenario'],
           y=comparacion_df['Unidades_Transferidas'],
           name='Unidades',
           marker_color='teal'),
        row=2, col=2
    )

fig.update_layout(
    height=800,
    showlegend=True,
    title_text="Análisis Comparativo de Escenarios de Optimización"
)

fig.update_xaxes(title_text="Escenario", row=1, col=1)
fig.update_xaxes(title_text="Escenario", row=1, col=2)
fig.update_xaxes(title_text="Escenario", row=2, col=1)
fig.update_xaxes(title_text="Escenario", row=2, col=2)

fig.update_yaxes(title_text="Costo ($)", row=1, col=1)
fig.update_yaxes(title_text="Costo ($)", row=1, col=2)
fig.update_yaxes(title_text="Cantidad", row=2, col=1)
fig.update_yaxes(title_text="Unidades", row=2, col=2)

fig.show()

```

Paso 9: Exportar Resultados Detallados

Guardaremos las transferencias óptimas de cada escenario.

```

In [20]: # Consolidar todos los resultados en un único DataFrame
todas_transferencias = []

for escenario, df_trans in resultados_por_escenario.items():
    if len(df_trans) > 0:
        df_temp = df_trans.copy()
        df_temp['Escenario'] = escenario
        todas_transferencias.append(df_temp)

if todas_transferencias:
    transferencias_consolidadas = pd.concat(todas_transferencias, ignore_index=True)
    transferencias_consolidadas = transferencias_consolidadas[[
        'Escenario', 'Tienda_Origen', 'Tienda_Destino', 'SKU', 'Cantidad', 'Costo_Transferencia'
    ]]

    # Guardar
    transferencias_consolidadas.to_csv('transferencias_optimas.csv', index=False)
    print("Transferencias óptimas guardadas en 'transferencias_optimas.csv'")
    print(f" Total de registros: {len(transferencias_consolidadas)}")

    # Mostrar muestra
    print("\nMUESTRA DE TRANSFERENCIAS ÓPTIMAS:")
    print(transferencias_consolidadas.head(45).to_string(index=False))
else:
    print("No se generaron transferencias en ningún escenario")

```

Transferencias óptimas guardadas en 'transferencias_optimas.csv'
Total de registros: 45

MUESTRA DE TRANSFERENCIAS ÓPTIMAS:

	Escenario	Tienda_Origen	Tienda_Destino	SKU	Cantidad	Costo_Transferencia
	Escenario 1	TIENDA005	TIENDA003	HM000003	10	30.0
	Escenario 1	TIENDA001	TIENDA005	HM000062	3	9.0
	Escenario 1	TIENDA001	TIENDA005	HM000005	2	6.0
	Escenario 1	TIENDA001	TIENDA003	HM000005	19	57.0
	Escenario 1	TIENDA003	TIENDA005	HM000035	7	21.0
	Escenario 1	TIENDA003	TIENDA005	HM000057	14	42.0
	Escenario 1	TIENDA003	TIENDA001	HM000035	10	30.0
	Escenario 1	TIENDA003	TIENDA001	HM000057	10	30.0
	Escenario 1	TIENDA003	TIENDA001	HM000041	14	42.0
	Escenario 2	TIENDA005	TIENDA003	HM000003	10	40.0
	Escenario 2	TIENDA001	TIENDA005	HM000062	3	12.0
	Escenario 2	TIENDA001	TIENDA005	HM000005	2	8.0
	Escenario 2	TIENDA001	TIENDA003	HM000005	19	76.0
	Escenario 2	TIENDA003	TIENDA005	HM000035	7	28.0
	Escenario 2	TIENDA003	TIENDA005	HM000057	14	56.0
	Escenario 2	TIENDA003	TIENDA001	HM000035	10	40.0
	Escenario 2	TIENDA003	TIENDA001	HM000057	10	40.0
	Escenario 2	TIENDA003	TIENDA001	HM000041	14	56.0
	Escenario 3	TIENDA005	TIENDA003	HM000003	10	50.0
	Escenario 3	TIENDA001	TIENDA005	HM000062	3	15.0
	Escenario 3	TIENDA001	TIENDA005	HM000005	2	10.0
	Escenario 3	TIENDA001	TIENDA003	HM000005	19	95.0
	Escenario 3	TIENDA003	TIENDA005	HM000035	7	35.0
	Escenario 3	TIENDA003	TIENDA005	HM000057	14	70.0
	Escenario 3	TIENDA003	TIENDA001	HM000035	10	50.0
	Escenario 3	TIENDA003	TIENDA001	HM000057	10	50.0
	Escenario 3	TIENDA003	TIENDA001	HM000041	14	70.0
	Escenario 4	TIENDA005	TIENDA003	HM000003	10	60.0
	Escenario 4	TIENDA001	TIENDA005	HM000062	3	18.0
	Escenario 4	TIENDA001	TIENDA005	HM000005	2	12.0
	Escenario 4	TIENDA001	TIENDA003	HM000005	19	114.0
	Escenario 4	TIENDA003	TIENDA005	HM000035	7	42.0
	Escenario 4	TIENDA003	TIENDA005	HM000057	14	84.0
	Escenario 4	TIENDA003	TIENDA001	HM000035	10	60.0
	Escenario 4	TIENDA003	TIENDA001	HM000057	10	60.0
	Escenario 4	TIENDA003	TIENDA001	HM000041	14	84.0
	Escenario 5	TIENDA005	TIENDA003	HM000003	10	70.0
	Escenario 5	TIENDA001	TIENDA005	HM000062	3	21.0
	Escenario 5	TIENDA001	TIENDA005	HM000005	2	14.0
	Escenario 5	TIENDA001	TIENDA003	HM000005	19	133.0
	Escenario 5	TIENDA003	TIENDA005	HM000035	7	49.0
	Escenario 5	TIENDA003	TIENDA005	HM000057	14	98.0
	Escenario 5	TIENDA003	TIENDA001	HM000035	10	70.0
	Escenario 5	TIENDA003	TIENDA001	HM000057	10	70.0
	Escenario 5	TIENDA003	TIENDA001	HM000041	14	98.0

Paso 10: Análisis Detallado de un Escenario Específico

Analicemos en detalle el mejor escenario.

```
In [21]: # Analizar el mejor escenario
escenario_analizar = mejor_escenario
transferencias_escenario = resultados_por_escenario[escenario_analizar]

print(f"\n{'='*80}")
print(f"ANÁLISIS DETALLADO: {escenario_analizar}")
print(f"{'='*80}")

if len(transferencias_escenario) > 0:
    # Análisis por tienda origen
    print("\nTIENDAS ORIGEN (Proveedoras):")
    origenes = transferencias_escenario.groupby('Tienda_Origen').agg({
        'Cantidad': 'sum',
        'Costo_Transferencia': 'sum'
    }).sort_values('Cantidad', ascending=False)
    print(origenes)

    # Análisis por tienda destino
    print("\nTIENDAS DESTINO (Receptoras):")
    destinos = transferencias_escenario.groupby('Tienda_Destino').agg({
        'Cantidad': 'sum',
        'Costo_Transferencia': 'sum'
    }).sort_values('Cantidad', ascending=False)
    print(destinos)

    # Análisis por SKU
    print("\nSKUs MÁS TRANSFERIDOS:")
    skus_trans = transferencias_escenario.groupby('SKU').agg({
        'Cantidad': 'sum',
        'Costo_Transferencia': 'sum'
    }).sort_values('Cantidad', ascending=False)
```



```
print(skus_trans)

# Crear visualización de flujo de transferencias
print(f"\nTODAS LAS TRANSFERENCIAS:")
print(transferencias_escenario.to_string(index=False))

else:
    print("\nNo se recomiendan transferencias en este escenario")
    print("    Esto puede indicar que los stocks actuales son adecuados")
```

ANÁLISIS DETALLADO: Escenario 1

TIENDAS ORIGEN (Proveedoras):

	Cantidad	Costo_Transferencia
Tienda_Origen		
TIENDA003	55	165.0
TIENDA001	24	72.0
TIENDA005	10	30.0

TIENDAS DESTINO (Receptoras):

	Cantidad	Costo_Transferencia
Tienda_Destino		
TIENDA001	34	102.0
TIENDA003	29	87.0
TIENDA005	26	78.0

SKUs MÁS TRANSFERIDOS:

	Cantidad	Costo_Transferencia
SKU		
HM000057	24	72.0
HM000005	21	63.0
HM000035	17	51.0
HM000041	14	42.0
HM000003	10	30.0
HM000062	3	9.0

TODAS LAS TRANSFERENCIAS:

Tienda_Origen	Tienda_Destino	SKU	Cantidad	Costo_Transferencia
TIENDA005	TIENDA003	HM000003	10	30.0
TIENDA001	TIENDA005	HM000062	3	9.0
TIENDA001	TIENDA005	HM000005	2	6.0
TIENDA001	TIENDA003	HM000005	19	57.0
TIENDA003	TIENDA005	HM000035	7	21.0
TIENDA003	TIENDA005	HM000057	14	42.0
TIENDA003	TIENDA001	HM000035	10	30.0
TIENDA003	TIENDA001	HM000057	10	30.0
TIENDA003	TIENDA001	HM000041	14	42.0